

Spring MVC Basics

Q1. What is Spring MVC, and what problem does it solve in web application development?

A: **Spring MVC (Model-View-Controller)** is a framework built on top of the core Spring Framework for developing web applications. It solves the problem of organizing web code by separating concerns into three distinct roles: the **Model** (data), the **View** (user interface), and the **Controller** (business logic). This separation makes applications easier to develop, test, and maintain.

Q2. What are the main components of the Spring MVC architecture?

A: The main component is the **DispatcherServlet**, which acts as the front controller, handling all incoming requests. Other key components include **Controllers** (process requests), **ViewResolvers** (select the view), and **Models** (pass data to the view). The DispatcherServlet coordinates the interaction between all these parts.

Controller & Request Mapping

Q3. What is the purpose of the @RequestMapping annotation in Spring MVC?

A: The purpose of the **@RequestMapping** annotation is to map incoming HTTP requests to specific handler methods or classes within your application. It uses attributes like URL path, HTTP method, and consumed/produced media types to match the request to the correct code.

Q4. How do you define method-level request mappings within a controller class?

A: You define method-level mappings by placing the **@RequestMapping** annotation directly above the method signature. Alternatively, you can use specialized shortcut annotations like **@GetMapping**, **@PostMapping**, or **@DeleteMapping** for better clarity and conciseness.

Q5. What attributes can be used inside @RequestMapping, and what do they control?

A: Key attributes include **value** or **path** (controls the URL), **method** (controls the HTTP verb like GET or POST), **consumes** (controls the expected media type of the request body), and **produces** (controls the expected media type of the response body). These attributes help differentiate and route traffic precisely.

Q6. How does **@RequestMapping** differentiate between various HTTP methods like GET, POST, PUT, DELETE?

A: The **@RequestMapping** annotation uses the **method** attribute to specify the required HTTP verb (e.g., **@RequestMapping(method = RequestMethod.GET)**). Today, developers use the dedicated annotations like **@GetMapping**, **@PostMapping**, and **@DeleteMapping** which are cleaner shortcuts that handle the method differentiation automatically.

Controller vs RestController

Q7. What is the difference between **@Controller** and **@RestController**?

A: **@Controller** is a general-purpose annotation used when the handler method typically returns a **View Name** (like an HTML file). **@RestController** is a convenience annotation that combines **@Controller** and **@ResponseBody**. This means methods automatically return data (like JSON or XML) directly in the HTTP response body.

Q8. In which situations would you choose **@RestController** instead of **@Controller**?

A: You would choose **@RestController** when building a **REST API** intended to be consumed by other applications, mobile clients, or JavaScript frontends. You choose **@Controller** only when you are building a traditional, server-side rendered application that returns an actual web page (a View).

Q9. How does the response handling differ between MVC controllers and REST controllers?

A: In a standard **MVC controller**, the method return value is usually treated as a logical view name, which the **ViewResolver** converts into a template path. In a **REST controller**, the return value is immediately serialized (converted) into a data format (JSON/XML) and written directly into the HTTP response body.

Q10. What happens internally with data serialization when using **@RestController**?

A: When using **@RestController**, Spring uses a **MessageConverter** (usually Jackson) to serialize the returned Java object. The converter automatically transforms the Java object into a string format (JSON or XML) based on the client's requested content type and places that serialized data directly into the response body.

Form Handling & Data Binding

Q11. How do you handle HTML form data in a Spring MVC application?

A: You handle form data by creating a **POJO (Plain Old Java Object)** to represent the form fields. The handler method accepts this POJO as an argument, and Spring automatically binds the data submitted from the HTML form's input fields into the corresponding fields of the POJO.

Q12. How do you process form submissions and map them to model objects?

A: You process submissions by defining a controller method that maps to the form's action URL and uses the **POST** method. The method accepts the form data object (DTO/POJO) as a parameter, and Spring automatically maps the incoming key-value pairs from the request to the object's fields.

Q13. What is the purpose of @ModelAttribute, and how does it assist with form binding?

A: The purpose of **@ModelAttribute** is to bind form data submitted via HTTP POST/GET requests to a Java object. It assists binding by first creating the object and then populating its fields using matching request parameter names. It also adds this object to the Model for view rendering.

Q14. How does Spring MVC support automatic data binding for request parameters?

A: Spring MVC supports automatic data binding through the **WebDataBinder** component. It uses naming conventions to match request parameter names (e.g., userName=John) to the properties of a method argument object, performing type conversion automatically where needed.

Validation

Q15. How do you implement form validation in Spring MVC?

A: You implement validation by using annotations from the **Bean Validation API** (like **@NotNull**, **@Size**, **@Email**) on the fields of your form backing object. The validation process is triggered by placing the **@Valid** or **@Validated** annotation next to the form object parameter in the controller method.

Q16. What steps are required to enable validation with annotations like `@Valid` or `@Validated`?

A: You need three steps: 1) Ensure you have the **Bean Validation API** implementation (like Hibernate Validator) on your classpath. 2) Annotate the fields of your POJO with validation rules. 3) Place `@Valid` or `@Validated` immediately before the model object in the controller method signature.

Views & ViewResolvers

Q17. What is a `ViewResolver` in Spring MVC, and why is it needed?

A: A **`ViewResolver`** is a component that translates a logical view name (a simple string returned by a controller, like "home") into the actual, physical view technology (like a file path to a JSP or a Thymeleaf template). It is needed because it decouples the controller logic from the specific rendering technology used.

Q18. What different types of `ViewResolvers` can be used in a Spring MVC application?

A: Common types include **`InternalResourceViewResolver`** (used for JSPs and static HTML files), **`UrlBasedViewResolver`** (for generic URLs), and specialized resolvers for template engines like **Thymeleaf** or **FreeMarker**. You can configure a chain of multiple `ViewResolvers` to support different view technologies.

Q19. How does `InternalResourceViewResolver` resolve JSP or template files?

A: **`InternalResourceViewResolver`** resolves files by using a configurable prefix and suffix. For example, if configured with a prefix of `/WEB-INF/views/` and a suffix of `.jsp`, the logical view name "home" is resolved to the physical file path `/WEB-INF/views/home.jsp`.

Interceptors & Filters (Intermediate Level)

Q20. What are interceptors in Spring MVC, and what use-cases do they address?

A: Interceptors are objects that allow you to execute code *before* or *after* a controller method is handled. They address use cases like logging, auditing, checking user permissions, and manipulating the Model before rendering the View. They are designed to handle cross-cutting concerns globally.

Q21. Which methods does the `HandlerInterceptor` interface provide, and when are they executed?

A: The `HandlerInterceptor` interface provides three methods: **`preHandle()`** (executed *before* the controller method), **`postHandle()`** (executed *after* the controller method but *before* the view is rendered), and **`afterCompletion()`** (executed *after* the complete request, including view rendering, useful for resource cleanup).

Q22. How does a Spring MVC interceptor differ from a servlet filter?

A: A **Servlet Filter** is part of the servlet specification and runs *before* the `DispatcherServlet`, acting on all requests regardless of the handler. A **Spring MVC Interceptor** runs *after* the `DispatcherServlet` has determined the handler, giving it access to the handler object, the Model, and the specific execution context within Spring.

Exception Handling

Q23. How do you handle exceptions in Spring MVC?

A: You handle exceptions by centralizing error logic using an **`@ControllerAdvice`** class. This class acts as a global handler for exceptions thrown by any controller. It can catch exceptions and return a standard, predictable error response (like JSON) to the client.

Q24. What is the role of `@ExceptionHandler` within controllers?

A: The role of **`@ExceptionHandler`** is to define a local method that handles specific exceptions thrown only within that single controller class. It allows the controller to return a custom response (a view or a JSON body) tailored to that specific error, ensuring the application fails gracefully.

Dependency Injection in MVC

Q25. What does dependency injection mean in the context of Spring MVC?

A: Dependency injection means that the Spring container automatically provides an object with the other objects (dependencies) it needs to perform its job. In MVC, this typically involves the container providing a Controller with its required **Service** or **Repository** components.

Q26. How are dependencies injected into controllers?

A: Dependencies are injected into controllers using **Autowiring**. The controller class marks a field or constructor with the **`@Autowired`** annotation. The Spring container then finds the matching dependency bean and supplies it to the controller when the controller itself is instantiated.

Q27. What types of DI are supported (constructor, setter, field)?

A: Spring supports three types of dependency injection: **Constructor Injection** (via the class constructor), **Setter Injection** (via public setter methods), and **Field Injection** (directly on the private field). Constructor injection is generally preferred as it guarantees all dependencies are available upon object creation.

Q28. Why is dependency injection useful in MVC applications?

A: DI is useful because it promotes **loose coupling**. It allows you to easily swap implementations (e.g., replace a real database service with a mock service for testing) without changing the Controller code. This makes the application highly modular, flexible, and much easier to unit test.

Request Parameters & Path Variables

Q29. What is the use of @RequestParam, and when do you choose it?

A: **@RequestParam** is used to extract values from the URL's **query parameters** (e.g., ?id=123). You choose it when you need to handle optional data, filtering criteria, or sorting parameters that do not identify a specific resource.

Q30. What is @PathVariable, and how do you extract dynamic values from the URL using it?

A: **@PathVariable** is used to extract dynamic values that are part of the actual **URL path**. You extract the value by defining a placeholder in the mapping URL (e.g., /users/{id}) and then marking the corresponding method parameter with **@PathVariable("id")**. This value typically identifies the specific resource being accessed.

Static Resources

Q31. How do you configure Spring MVC to serve static files such as CSS, JS, or images?

A: You configure static files by adding a resource handler mapping in your configuration. You use **addResourceHandler("/resources/**")** to map the external path and **addResourceLocations("/static/")** to specify the internal classpath directory (like src/main/resources/static) where the files are actually located.

Testing Spring MVC

Q32. How do you test Spring MVC controllers and endpoints? (MockMvc)

A: You test controllers and endpoints using Spring's **MockMvc** utility, typically along with the **@WebMvcTest** annotation. MockMvc allows you to send mock HTTP requests (GET, POST,

etc.) and assert the response status, headers, and content without needing to start a full servlet container.

Q33. What frameworks are commonly used to test Spring MVC components?

A: The core testing framework is **JUnit 5**. **Mockito** is used to create mock dependencies (Services, Repositories). The **@WebMvcTest** annotation from Spring Boot is used to load only the necessary web layer components for fast, focused slice tests.

JAVA Interview Buddy